AI Decathlon Predictor

Informační technologie - ML/AI projekt

Ondřej Krejčí

krejci3@spsejecna.cz

Úvod

Tato dokumentace popisuje webovou aplikaci AI Decathlon Predictor. Cílem tohoto softwaru je poskytnout sportovcům, trenérům a fanouškům nástroj pro přesnou predikci celkového bodového zisku v desetiboji. Výpočet probíhá dynamicky na základě výkonů, které uživatel zadá do minimálně pěti libovolně zvolených disciplín.

Jádrem aplikace je regresní model využívající skupinové učení. Ten nepočítá výsledek jedním algoritmem, ale průměruje predikce deseti nezávislých modelů, čímž se minimalizuje chybovost. Trénovací data jsem získal web scrapingem. Více než 2000 reálných desetibojů z portálu *decathlon2000.com* jsem algoritmicky vyčistil od nedokončených závodů a statistických anomálií.

Celý systém je programátorsky postavený na architektonickém návrhovém vzoru MVC, který bezpečně odděluje uživatelské rozhraní od složité výpočetní logiky na pozadí. Následující stránky tohoto dokumentu slouží jako komplexní průvodce celým mým projektem.

Obsah

AI Decathlon Predictor **1**
Úvod 2
Obsah 3
Architektura aplikace 4
Komponenty systému a jejich vazby: 4
1. Model - src/data_loader.py: 4
2. Controller - src/ml_engine.py: 4
3. View - src/view.py: 4
Závislosti a knihovny třetích stran 6
Sběr a zpracování dat 7
Instalace a spuštění aplikace 8
Systémové požadavky: 8
Postup instalace a spuštění: 8
Konfigurace programu 9
Chybové stavy a jejich řešení 10
Testování 11

Architektura aplikace

Aplikace je navržena podle architektonického návrhového vzoru **MVC**. Tento přístup striktně odděluje data, aplikační logiku a uživatelské rozhraní.

Komponenty systému a jejich vazby:

1. **Model - *src/data_loader.py*:**

Tato komponenta tvoří základnu pro veškeré operace strojového učení. Jejím úkolem je načtení a distribuce připravených dat.

2. **Controller - *src/ml_engine.py*:**

Řídicí vrstva funguje jako zprostředkovatel mezi daty a uživatelským rozhraním. Je to nejkomplexnější část systému, která pracuje s algoritmy umělé inteligence.

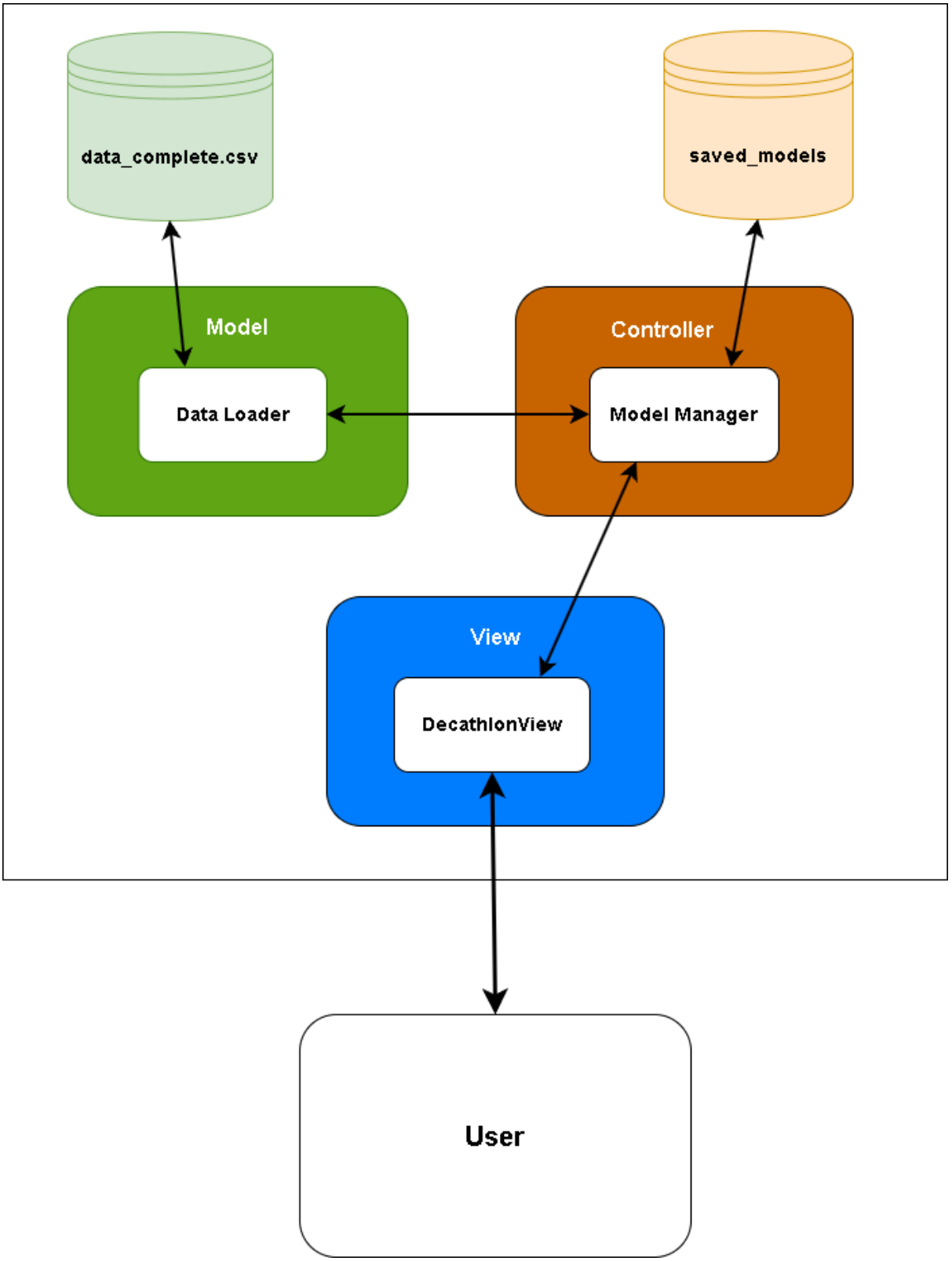
- **ModelManager:** Tato třída zapouzdřuje logiku pro správu regresních modelů. Rozhoduje o tom, zda je nutné spustit nový tréninkový proces, nebo lze využít již existující model z disku.
- **Ensemble Learning:** Pro minimalizaci chybovosti využívá systém metodu Bagging. Namísto jednoho algoritmu je paralelně spuštěno 10 nezávislých regresních modelů, přičemž každý je trénován na mírně odlišném podmnožině dat. Finální bodový zisk nevzniká výpočtem jednoho modelu, ale jako aritmetický průměr celé skupiny. Tento přístup dramaticky snižuje riziko extrémních chyb (outlierů) a garantuje vysoce robustní výsledek i při netypických uživatelských vstupech.
- **Persistence dat:** Pro optimalizaci výkonu jsou natrénované skupiny modelů serializovány do binárního formátu pomocí knihovny Joblib a ukládány do složky *saved_models*. To umožňuje okamžitou odezvu aplikace při dříve použitém výběru kombinace disciplín.

3. **View - *src/view.py*:**

Prezentační vrstva je realizována pomocí moderního frameworku Streamlit.

- **DecathlonView:** Třída definující layout aplikace. UI dynamicky reaguje na přidávání nebo odebírání disciplín v reálném čase.
- **Validace a parsování vstupů:** Klíčovou funkcí je inteligentní parser pro běh na 1500 metrů. Ten umožňuje uživateli zadat čas v přirozeném atletickém formátu (MM:SS.d), který následně transformuje na sekundy, se kterými pracuje matematické jádro.

- **Ošetření chybových stavů:** View vrstva zachycuje výjimky vzniklé při výpočtech a komunikuje je uživateli srozumitelnou formou, čímž zabraňuje pádu celé aplikace při zadání nesmyslných hodnot.



Závislosti a knihovny třetích stran

Aplikace je napsána v jazyce Python (verze 3.9+) a je závislá na následujících rozhraních a knihovnách:

- **Streamlit:** Tato knihovna je nasazena v souborech *app.py* a *src/view.py*, kde plní roli kompletního webového serveru a frontendového frameworku. Stará se o zachytávání uživatelských interakcí, jako je výběr disciplín a zadávání výkonů, a zajišťuje dynamické překreslování uživatelského rozhraní v prohlížeči, aniž by bylo nutné psát oddělený kód v HTML, CSS nebo JavaScriptu.
- **Scikit-Learn:** Modul *src/ml_engine.py* je pracuje převážně na této knihovně, jelikož se stará o samotnou inteligenci aplikace. Projekt z knihovny konkrétně využívá třídu GradientBoostingRegressor pro tvorbu a trénování samotných regresních modelů a funkci train_test_split, která se stará o bezpečné rozdělení dat na trénovací a testovací sady při generování finálního skupinového modelu.
- **Pandas:** Knihovna Pandas je klíčová pro datovou vrstvu aplikace a je použita primárně v *src/data_loader.py*. Zde se stará o načtení atletických dat ze souboru *data_complete.csv* do DataFrame struktury. Dále knihovna byla použita pro veškeré předzpracování, tedy filtraci prázdných hodnot, odstraňování DNF záznamů a finální přípravu strukturovaných dat pro algoritmy strojového učení.
- **NumPy:** Tato knihovna je využita v kontroleru *src/ml_engine.py* pro rychlé a optimalizované numerické výpočty. Její hlavní rolí v projektu je výpočet finálního konsenzu v rámci Ensemble modelu, kde se pomocí funkce np.mean stará o zprůměrování predikcí od deseti nezávislých regresních modelů do jednoho výsledného bodového odhadu.
- **Joblib:** Knihovna Joblib je implementována uvnitř *src/ml_engine.py*, kde se stará o inteligentní ukládání dat. Její rolí je serializovat natrénované seznamy modelů a ukládat je jako binární soubory do složky *saved_models*. Při opakovaném dotazu uživatele se stará o jejich okamžité načtení, čímž aplikaci šetří čas a výpočetní výkon, který by jinak byl potřeba k opětovnému tréninku.
- **Requests:** Tato knihovna figuruje v odděleném přípravném skriptu pro web scraping a nestartuje se při běžném běhu aplikace. Stará se o navazování spojení se servery portálu *decathlon2000.com*. Její rolí je bezpečně a efektivně stáhnout HTML kód webových stránek, na kterých se nacházejí historické výsledky desetibojů.
- **BeautifulSoup4:** Tato knihovna úzce spolupracuje s modulem Requests v rámci procesu sběru dat. Stará se o parsování staženého HTML kódu. V projektu je využita k algoritmickému procházení DOM stromu webové stránky, identifikaci specifických HTML tagů a k následné extrakci čistých datových bodů, jako jsou jména atletů, konkrétní disciplíny a jejich bodové zisky. Výstupem její práce je strukturovaný dataset, který je následně uložen do CSV formátu.

Sběr a zpracování dat

Zásadním předpokladem pro vytvoření kvalitního a přesného modelu strojového učení je zajištění dostatečně velkého a relevantního datasetu. Celkem jsem získal 2291 historických výsledků ze soutěží Götzis Hypomeeting, Talence Décastar, Ratingen Mehrkampf-Meeting, USA championships decathlon, Multistar, Kladno meeting, decathlon Alhama de murcia, Fred Kudu memorial, Estonian championshipps decathlon.

1. Získávání dat (Web Scraping)

Jako primární zdroj pro získání historických výsledků z reálných atletických soutěží jsem zvolil specializovaný portál *decathlon2000.com*. K automatizovanému stažení dat jsem vytvořil vlastní skript, který procházel HTML strukturu webu, vyhledával výsledkové tabulky a extrahoval z nich výkony jednotlivých atletů ve všech deseti disciplínách a jejich celkové bodové skóre. Tímto způsobem jsem shromáždil surová data.

2. Čištění a transformace (Data Cleaning)

Kvůli velkému množství chyb, překlepů a nekonzistencí, jsem musel data vyčistit. Při čistění jsem zahrnoval následující kroky:

- **Odstranění nekompletních záznamů:** Z datasetu byli algoritmicky vyřazeni všichni atleti, kteří závod nedokončili (DNF - Did Not Finish), nenastoupili (DNS), nebo v některé disciplíně nezaznamenali platný pokus (NM - No Mark).
- **Sjednocení datových typů a formátů:** Bylo nutné vyřešit nekonzistenci v zápisu běžeckých časů. Zejména u běhu na 1500 metrů byl textový formát (minuty a sekundy) pomocí vlastního parseru převeden na celkové sekundy v datovém typu float, aby s ním mohl matematický model pracovat.
- **Ošetření anomálií:** Datová sada byla zkontrolována na přítomnost extrémních statistických odchylek a zjevných překlepů na straně zdrojového webu (např. nemožné výkony neodpovídající lidským možnostem).

3. Výstupní dataset

Výsledkem této fáze je čistý, standardizovaný a plně validní soubor *data_complete.csv*. Tento soubor je uložen v adresářové struktuře projektu a slouží jako primární a jediný zdroj hodnot pro datovou vrstvu aplikace, ze které následně čerpají učící se algoritmy.

Instalace a spuštění aplikace

Software je koncipován pro snadné nasazení v lokálním prostředí bez nutnosti instalace a konfigurace komplexních webových serverů typu Apache nebo Nginx. O lokální hostování se stará přímo framework Streamlit.

Systémové požadavky:

- Operační systém: Windows 10/11, macOS nebo distribuce Linuxu.
- Programové vybavení: Nainstalovaný Python ve verzi 3.9 nebo vyšší.

Postup instalace a spuštění:

1) **Instalace a rozbalení:**

Zdrojové kódy aplikace jsou veřejně hostovány na platformě GitHub. Projekt si stáhne do svého počítače jedním z následujících způsobů:

- Pomocí stisknutí tlačítka *Code* a následně vybráním varianty *Download ZIP* na GitHub.
- Pomocí příkazové řádky a nástroje Git: *git clone https://github.com/ondrejkrejci1/ML_Project.git*

Následně rozbalte zip složku a ujistěte se, že se v podadresáři nachází soubor *data_complete.csv.*

2) **Instalace závislostí a knihoven třetích stran:**

Pro běh aplikace je nutné do systému doinstalovat požadované knihovny. V příkazovém řádku se přesuňte do hlavní složky aplikace a zadejte tento příkaz: *pip install pandas scikit-learn streamlit joblib numpy*

3) **Start webového serveru:**

Samotná aplikace se spouští pomocí nástroje Streamlit zadáním příkazu: *streamlit run app.py.* Opět platí, že se musíte nacházet v kořenovém adresáři projektu.

4) **Přístup do uživatelského rozhraní:**

Po úspěšném spuštění serveru v terminálu se aplikace automaticky otevře ve výchozím webovém prohlížeči. Pokud se tak nestane, můžete k ní přistoupit zadáním lokální adresy URL na port 8501 (*http://localhost:8501*).

Konfigurace programu

Aplikace je navržena tak, aby byla maximálně modulární a nevyžadovala tvrdý zásah do zdrojového kódu při změně základních parametrů. Pro tyto účely slouží centrální konfigurační soubor config.json, umístěný v kořenovém adresáři projektu.

Tento soubor definuje klíčové cesty a proměnné, které si při startu načítá datová vrstva i kontroler.

Typická struktura a volby souboru config.json:

- "data_path": Definuje relativní cestu ke zdrojovému souboru s daty (např. data/data_complete.csv). Umožňuje snadné přepojení aplikace na novější dataset, aniž by bylo nutné upravovat Python skripty.
- "models_dir": Určuje cílovou složku pro ukládání natrénovaných regresních modelů (např. saved_models/).
- "features": Obsahuje pole povolených vstupních atributů (desetibojařských disciplín).

Kromě výše zmíněné statické konfigurace disponuje aplikace také dynamickou modulací vstupů přímo za běhu programu. Uživatel není vázán na pevnou sadu dat, ale sám si v interaktivním rozhraní vybírá, které konkrétní atributy (disciplíny) pro výpočet použije. Řídicí vrstva systému na tuto uživatelskou volbu okamžitě reaguje, dynamicky přizpůsobuje strukturu dat a na pozadí trénuje model optimalizovaný výhradně pro tyto vybrané parametry.

Díky tomuto přístupu může uživatel snadno modifikovat chování aplikace pouhou úpravou textového JSON souboru, zatímco koncový uživatel má plnou kontrolu nad tvorbou predikčního modelu v reálném čase.

Chybové stavy a jejich řešení

Aplikace je navržena tak, aby byla odolná vůči nestandardnímu chování uživatele (tzv. idiot-proof design). Veškeré kritické operace jsou ošetřeny pomocí bloků try-except, aby se předešlo fatálnímu pádu celého systému.

Mezi hlavní identifikované chybové stavy a jejich řešení patří:

- **Neplatný formát času (1500 m):** Uživatel zadá čas ve špatném formátu (např. pouhý text místo MM:SS.d).
    - *Řešení:* Vlastní parser parse_time_to_seconds tuto chybu zachytí (ValueError), aplikace zobrazí červené varování s nápovědou správného formátu a pro výpočet dočasně dosadí bezpečnou výchozí hodnotu, aby nedošlo k pádu programu.
- **Pokus o výběr nepovoleného počtu disciplín:** Uživatel se pokusí odebrat disciplínu pod limit 5, nebo přidat nad limit 8.
    - *Řešení:* Uživatelské rozhraní tuto akci algoritmicky nedovolí a zobrazí žluté varování (st.warning) s vysvětlením pravidel projektu.
- **Chybějící zdrojová data:** Aplikace nenalezne soubor data_complete.csv v definované cestě.
    - *Řešení:* Vrstva DataLoader zachytí výjimku a předá ji kontroléru. Aplikace zobrazí uživateli zprávu, že pro trénink chybí data, a vyzve k nápravě adresářové struktury.

Testování

Abych zaručil bezchybný chod webové aplikace a její maximální odolnost vůči nestandardnímu chování uživatelů, podrobil jsem celý systém důkladnému manuálnímu testování. Pro tento účel jsem vypracoval sadu standardizovaných testovacích scénářů. Tyto scénáře záměrně nepokrývají pouze ideální průchod aplikací, ale zaměřují se i na dynamické změny parametrů modelu za běhu a na cílené vyvolání hraničních a chybových stavů. Tím je v praxi ověřena nejen přesnost samotného regresního výpočtu, ale také spolehlivost parserů a správné zachytávání výjimek (např. při zadání neplatného textového řetězce). Tím je v praxi ověřena správná funkčnost celé aplikace.

Detailní specifikace jednotlivých kroků, použitých testovacích dat a očekávaných výsledků je zaznamenána v následujících tabulkách v test_cases na GitHub: https://github.com/ondrejkrejci1/ML_Project/tree/main/test_cases.